

PART 1 — Classical Complexity

What is time complexity?

- A function that gives us the relationship about how the time will grow as the input grows

Now an example:

we have two Computers.
we run an algorithm. we have

old Computer

- data: 10,00,000 elements in array
- Algo: linear search
- for target that doesn't exit in the array
- Time taken: 10 sec

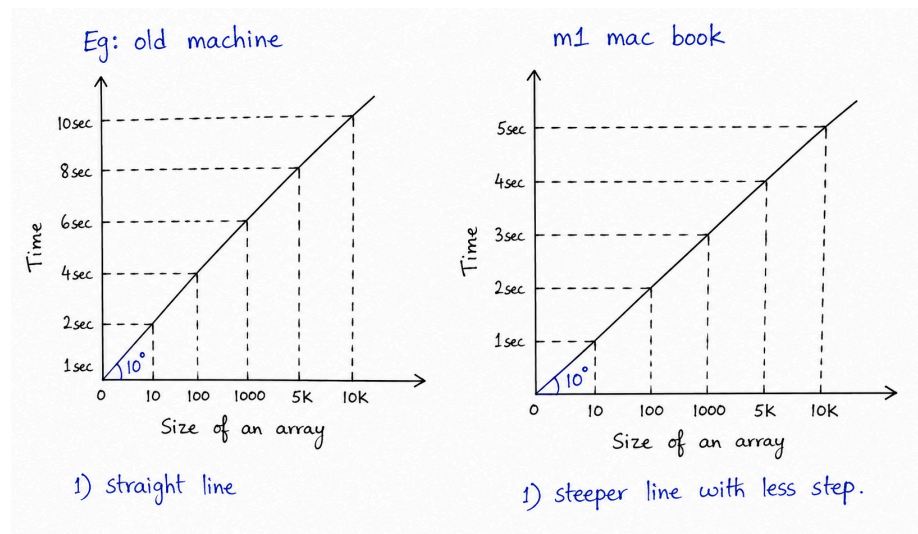
M1 macbook (very fast)

- data: 10,00,000 elements in array
- Algo: linear search
- for target that doesn't exit in the array
- Time taken: 1 sec.

Q. Which machine has a better time complexity in these?

Both of the machine have the same time complexity

Time Complexity != Time taken

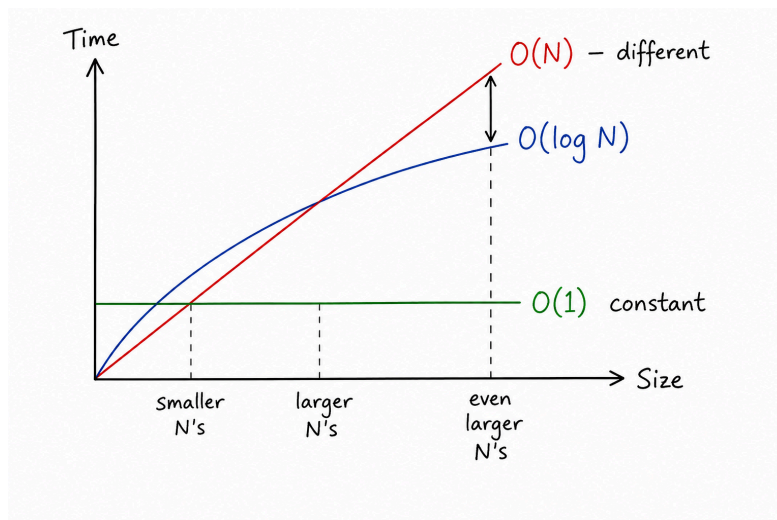


Even though the time taken is different but the relationship b/w the size and the "time" is same "linear"

Over here, time is growing linearly as the size is growing in both cases though values are different

Note:

1. In time complexity always looks at bigger numbers.
2. Always think about when your data will grow large in size, in that case what will happen?

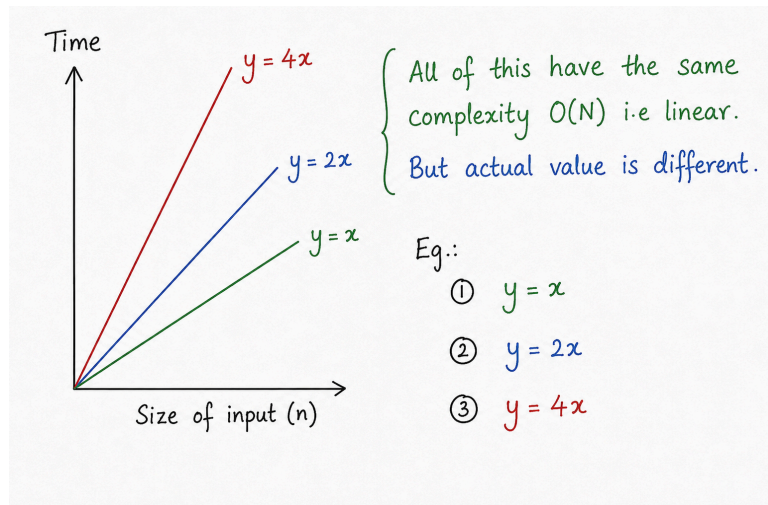


3. $O(1) < O(\log N) < O(N)$

- Now in the fig we can see that the (red) linear is taking the most time, then $\log(n)$, then constant.
- Therefore, constant is always better than $O(\log(n))$ & $O(N)$
- As you can see, when the size was fixed for the same amount of data, $O(N)$ was taking the most time, then $\log(N)$ & last $O(1)$.

Q. What do we consider when thinking about the complexity?

1. Always look for worst case complexity
2. Always look at complexity for large/infinite of data



a: Even though the value of actual time is different they're all growing linearly.

b: "we don't actually care about what the time taken is because that will vary from machine to machine" we only care about the relationship of how the time will grow when the input grows.

Q. Do we really need to worry about these constant then?

No, we only care about how it's growing.

C. This is why, we ignore all constants.

- Always ignore less dominating terms.

Big Oh Notation:

Def: suppose, that an algorithm has complexity of $O(N^3)$

Q. what does it mean in simple words?

So, in simple language it means that this is the upper bound.

Meaning: N^3 Size of the array will grow as the input grows in an (N^3) fashion.

But, **what does this Big Oh saying?**

It's saying that the complexity, the graph & relationship can't exceed N^3

For eg: your algo that you've written may be solved in a constant time, it may be solved in $O(N)$ time, or $O(\log N)$ Or $O(N^2)$ times etc.

But it will never exceed the time complexity relationship/function value more than N^3

It will never be like $O(N^4)$ Or $O(N^3 \log N)$ etc.
mathematical Notation:

If $f(n)=O(g(n))$

Then, $\lim_{n \rightarrow \infty} g(n)/f(n) < \infty$

Big Omega Notation

Def: Opposite of Big Oh notation

Suppose an algorithm has complexity of $\Omega(N^3)$

Q. what does it mean in simple form?

- This means that it will take atleast N^3 time complexity.
- So, this means it is the lower bound.
- It will take atleast N^3 it can also take $N^4, N^3 \log N$ or N^5 Etc.
- But, it will never be lesser than N^3
- Minimum N^3 time complexity will be required.

Maths:

$\lim_{n \rightarrow \infty} g(n)/f(n) > 0$

Big Theta Notation

What if an algorithm has lower & upper bound as $O(N^2)$ and $\Omega(N^2)$

It was not looking so good. That's why Big Theta Notation came.

It will have

$0 < \lim_{n \rightarrow \infty} g(n)/f(n) < \infty$

$\Rightarrow \Theta(N^2)$

It is basically combining lower & upper bound.

Little-oh Notation- $o(N)$

- As we know that the BigOh notation was giving the upper bound. This little-oh notation is also gonna give the upper bound only.
- But, what is the distance? This is not strict upper bound.
- It is a loose upper bound.

Big-oh

- If $f=O(g)$
- It means that the growth of (f) is no faster than (g)
- $\therefore f \leq g$

little-oh

- If $f=o(g)$
- it means that it is more like (f) is smaller than (g)
- $\therefore f < g$
- It is strictly slower than (g)

little omega

As we know Big Omega was giving the lower bound. This little omega is also going to give lower bound, but it will not be tight. So, it's gonna be like loosely lower bound.

Big Ω

- If $f=\Omega(g)$
- this means $f \geq g$ (lower bound)
- It is giving a lower bound. It can increase more than that.

little ω

- If $f=\omega(g)$
- it means $f > g$ (strictly greater)

PART 2 — Why Classical Computing Struggles

As input size grows, some problems become extremely hard for classical computers.

This happens because certain complexities grow very fast, like: $O(2^N)$ and $O(N!)$

Example

If: $N=10$

Then: $2^{10}=1024$ operations are needed.

But if: $N=100$ then: 2^{100} operations are needed, which is enormous.

Even the fastest classical computers struggle with such growth.

Important Point

Making hardware faster only improves performance by a constant factor.

But exponential complexity keeps growing rapidly as input increases.

So the real problem is not machine speed.

The real problem is:

how fast the computation grows with input size.

This limitation of classical computing is one reason why quantum computing becomes important.

PART 3 — Transition to Quantum

Until now, classical computers were trying to solve problems using:

- faster processors
- more memory
- better hardware

But for some problems, this is still not enough because the complexity grows too quickly.

So, instead of only increasing hardware speed, scientists asked:

“Can we reduce the growth of computation itself?”

This idea led to Quantum Computing.

Unlike classical computers that use bits: 0 or 1

quantum computers use qubits, which can exist in: 0, 1, or both simultaneously using a property called superposition.

Because of this, quantum computers can solve certain problems much more efficiently than classical computers.

PART 4 — Classical vs Quantum

Feature	Classical Computer	Quantum Computer
Basic unit	Bit: 0 or 1	Qubit: 0, 1, or both simultaneously

How it explores possibilities	One at a time	Many paths simultaneously
Special trick	Fast arithmetic	Superposition + interference
Where it helps most	General computing	Specific problems (factoring, search, simulation)

Shor's Algorithm — An Exponential Speedup

From here the content in the course is good